



LINSORA

Plataforma de Gestão Financeira Pessoal Inteligente

DOCUMENTAÇÃO TÉCNICA DO SISTEMA

Especificação Arquitetural Completa & Guia de Engenharia

Este documento contém o detalhamento da arquitetura de software, estrutura de dados, fluxos de autenticação híbrida (Google Auth Native / GIS / Supabase OAuth), esquema de banco de dados PostgreSQL com RLS, organização modular do frontend Vanilla JS e guia de build nativo Android via Capacitor 8.

Versão do Sistema: 1.0.0
(MVP)

Autor: Equipe de Engenharia & Arquitetura
LINSORA

Data de Emissão: 08 de Agosto de
2026

1. Visão Geral do Sistema

O **LINSORA Finances** é uma solução moderna de gestão financeira pessoal desenvolvida com foco em alta performance, usabilidade responsiva e funcionamento híbrido (Web PWA e Android Nativo). O sistema permite o controle consolidado de contas bancárias, cartões de crédito, chaves Pix, planejamento de metas financeiras, controle de contas fixas e registro detalhado de transações (receitas e despesas).

Proposta de Valor & Diferenciais Técnicos:

- Arquitetura Offline-First Resiliente:** Funcionalidade completa via `LocalStorage` com sincronização em nuvem automática e transparente quando conectado ao `Supabase Cloud`.
- Autenticação Nativa de Alta Fidelidade:** Suporte ao seletor nativo do Android (Google Play Services / Credential Manager) integrado via Capacitor GoogleAuth plugin com rotas de fallback resilientes (GIS e Supabase OAuth).
- Segurança Extrema (Zero Data Leak):** Políticas estritas de Row Level Security (RLS) no PostgreSQL do Supabase, garantindo isolamento total por `auth.uid()` em todas as tabelas.
- Design System Premium Vanilla:** Interface responsiva desenvolvida em CSS3 moderno com suporte a tema escuro/claro, glassmorphism e micro-animações.

1.1 Casos de Uso Principais

MÓDULO / FUNCIONALIDADE	DESCRIÇÃO TÉCNICA	FLUXO DE DADOS
Autenticação Híbrida	Login por e-mail/senha, Google Native SSO (Android) ou Google GIS (Web). Criação automática de perfil no primeiro acesso.	Capacitor Plugin → Supabase Auth → Trigger PostgreSQL
Dashboard Consolidado	Visão panorâmica de saldo total, receitas/despesas do mês, atalhos rápidos e gráficos analíticos.	LinsoraStore → Renderização DOM → Chart.js
Gestão de Contas e Cartões	Cadastro, edição e exclusão de contas correntes e cartões de crédito com cálculo automático de limite usado e faturas.	CRUD LinsoraStore → Cloud Sync (Supabase <code>accounts</code> e <code>cards</code>)
Lançamento de Transações	Registro de receitas e despesas com categoria, data, forma de pagamento e parcelamento.	Validation → LocalStorage → Supabase <code>transactions</code>
Metas e Contas Fixas	Acompanhamento de progresso de economias e alerta de vencimento de contas fixas recorrentes.	Notifications Engine → Visual Progress Renderer

2. Stack Tecnológica & Dependências

Frontend Web Core

HTML5 Semântico, Vanilla JavaScript (ES6+ Modules & Classes), CSS3 Custom Properties (Design System sem frameworks CSS genéricos).

Camada Nativa Mobile

Capacitor 8 Core & CLI (`@capacitor/core` , `@capacitor/android` v8.5.0), Android SDK, Gradle 8.14, Java 17 JDK.

Backend & Banco de Dados

Supabase Cloud (PostgreSQL 15), Supabase JS Client v2, Supabase Auth Service, Row Level Security (RLS) & Triggers PL/pgSQL.

Autenticação & Plugins

`@codetrix-studio/capacitor-google-auth` (v3.4.0-rc.4), Google Identity Services (GIS JS SDK), Supabase OAuth Redirect.

Gráficos & Utilitários

Chart.js v4 (Renderização Canvas), FontAwesome 6 (Ícones), Google Fonts (Inter / Outfit), ServiceWorker PWA.

Testes & Automação

Playwright E2E Test Suite (`@playwright/test` v1.40.0), PowerShell Scripting para Server Local e Gradle Builder.

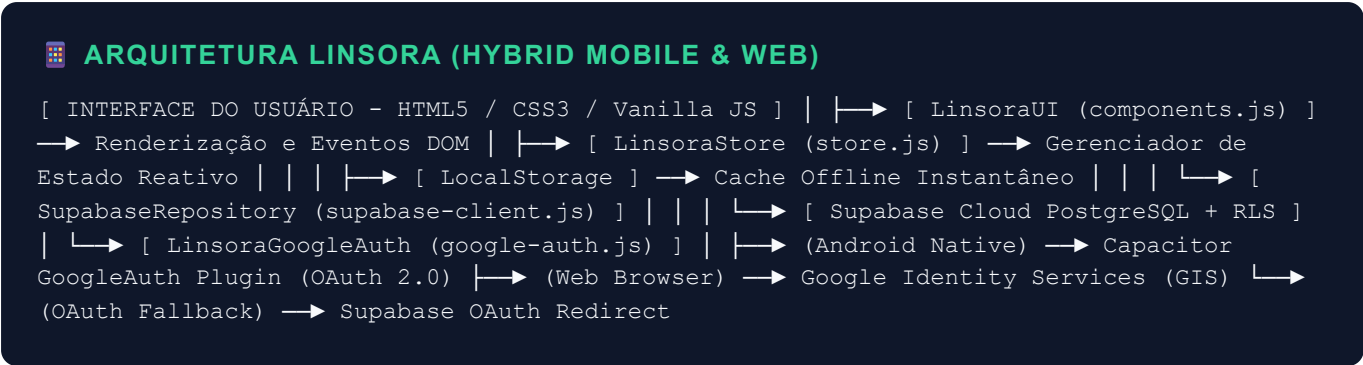
2.1 Configuração de Dependências (package.json)

```
{
  "name": "linsora-finance-mvp",
  "version": "1.0.0",
  "dependencies": {
    "@capacitor/android": "^8.5.0",
    "@capacitor/cli": "^8.5.0",
    "@capacitor/core": "^8.5.0",
    "@codetrix-studio/capacitor-google-auth": "^3.4.0-rc.4"
  },
  "devDependencies": {
    "@playwright/test": "^1.40.0"
  }
}
```

3. Arquitetura e Fluxos do Sistema

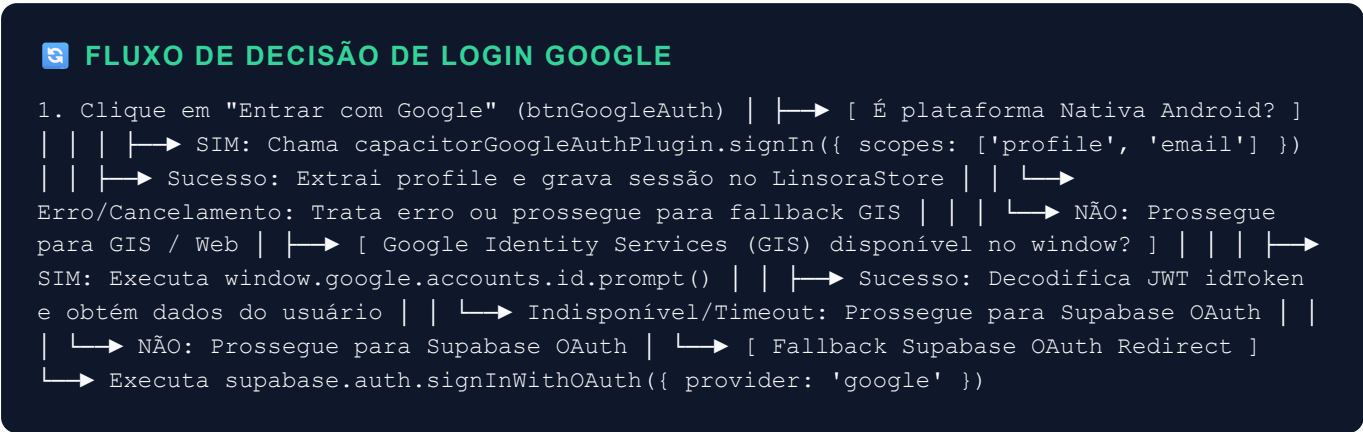
O LINSORA utiliza uma arquitetura em camadas bem delimitada. O frontend interage com os módulos internos de estado (`LinsoraStore`), que por sua vez realizam o roteamento de persistência entre a camada local (`LocalStorage`) e o repositório remoto (`SupabaseRepository`).

3.1 Diagrama Geral da Arquitetura



3.2 Fluxo de Autenticação Google Híbrido (Resiliente)

Para garantir 100% de disponibilidade em qualquer ambiente (Android nativo ou navegadores Web), o módulo `google-auth.js` implementa um mecanismo de tentativa e fallback em 3 camadas:



4. Estrutura de Banco de Dados e Segurança (PostgreSQL / RLS)

O modelo relacional do LINSORA foi projetado para alta escalabilidade e isolamento estrito de dados. Todas as tabelas possuem `Row Level Security (RLS)` ativado, permitindo que cada usuário acesse exclusivamente seus próprios registros baseando-se no token JWT validado pelo `auth.uid()`.

4.1 Dicionário de Dados

TABELA	COLUNA PRINCIPAL	TIPO	REGRA DE CHAVE ESTRANGEIRA / RLS
profiles	id	UUID (PK)	REFERENCES auth.users(id) ON DELETE CASCADE . Isolado por <code>auth.uid() = id</code> .
accounts	id, user_id	UUID (PK, FK)	Contas Bancárias. FK para <code>auth.users(id)</code> . RLS: <code>auth.uid() = user_id</code> .
cards	id, user_id	UUID (PK, FK)	Cartões de Crédito (limites e vencimentos). RLS: <code>auth.uid() = user_id</code> .
pix_keys	id, user_id	UUID (PK, FK)	Chaves Pix cadastradas (CPF, Email, Telefone, Aleatória).
goals	id, user_id	UUID (PK, FK)	Metas financeiras de economia com valor alvo e valor atual.
fixed_bills	id, user_id	UUID (PK, FK)	Contas fixas mensais recorrentes com dia de vencimento.
transactions	id, user_id	UUID (PK, FK)	Lançamentos de Receitas e Despesas com status e categorias.

4.2 Trigger Autônomo de Criação de Perfil

```
-- Trigger PL/pgSQL executado automaticamente ao registrar novo usuário no Supabase Auth
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.profiles (id, full_name, email, avatar_url)
    VALUES (
        NEW.id,
        COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.raw_user_meta_data->>'name', split_part(NEW.email, '@', 1)),
        COALESCE(NEW.raw_user_meta_data->>'avatar_url', 'https://images.unsplash.com/photo-1534528741775-
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

5. Estrutura de Pastas e Módulos do Código

O projeto segue uma estrutura organizacional desacoplada, separando recursos estáticos, lógicas de negócios, artefatos de compilação nativa e arquivos de teste.

```
FINANCEMVP/
├─ android/                                # Projeto Android Nativo (Gradle / Android Studio)
│   └─ app/src/main/
│       └─ java/com/linsora/app/          # MainActivity.java (Registro de Plugins Capacitor)
│           └─ AndroidManifest.xml        # Permissões da Internet e configurações de app
│       └─ gradlew.bat                    # Gradle Wrapper para automação de builds
├─ css/
│   └─ styles.css                        # Design System completo e estilos globais (70KB)
├─ js/
│   └─ app.js                            # Roteador principal, rotinas de inicialização e eventos
│   └─ store.js                          # LinsoraStore: Gestor de estado reativo e LocalStorage
│   └─ supabase-client.js                # Client Supabase e camada de sincronização em nuvem
│   └─ google-auth.js                    # LinsoraGoogleAuthManager (Android Native & GIS SSO)
│   └─ components.js                     # LinsoraUI: Componentes reutilizáveis, Modais e Toasts
│   └─ utils.js                           # Funções utilitárias (Formatação BRL, Datas, Export PDF/CSV)
│   └─ charts.js                         # Renderizador de gráficos interativos com Chart.js
│   └─ logger.js                         # Módulo central de logs e depuração do sistema
│   └─ notifications.js                  # Notificações visuais e agendamento de alertas
├─ tests/                                # Suite de Testes Automatizados E2E (Playwright)
│   └─ 04_accounts_cards.spec.js         # Testes de integração de contas e cartões
├─ www/                                  # Diretório de distribuição compilado para o Capacitor
├─ index.html                            # Layout principal Single Page Application (SPA)
├─ capacitor.config.json                 # Configuração global de IDs e Plugins do Capacitor
├─ supabase_schema_rls.sql               # Script SQL oficial com tabelas, RLS e Triggers
├─ server.ps1                            # Servidor HTTP local para desenvolvimento (PowerShell)
├─ package.json                          # Gerenciador de dependências e scripts npm
└─ LINSORA-Financas.apk                  # Pacote APK final de diagnóstico compilado
```

5.1 Responsabilidade dos Módulos JavaScript

MÓDULO JS	TAMANHO	PRINCIPAIS RESPONSABILIDADES
<code>app.js</code>	~42 KB	Controle de fluxo de telas, binding de formulários, modais e inicialização de instâncias.
<code>components.js</code>	~26 KB	Renderização dinâmica de tabelas de transações, cards de saldo, toasts e modais.
<code>store.js</code>	~19 KB	Gerenciamento de estado reativo em memória com sincronização bidirecional no <code>LocalStorage</code> .
<code>supabase-client.js</code>	~18 KB	Comunicação REST/Realtime com Supabase, fallback gracioso offline e hash de IDs.
<code>google-auth.js</code>	~12 KB	Gerenciamento de OAuth nativo Google (Play Services) e web Identity Services (GIS).
<code>utils.js</code>	~10 KB	Formatação de moeda BRL, formatação de datas ISO, máscaras de input e geradores de ID.

6. Guia de Configuração, Compilação e Build

6.1 Pré-requisitos do Ambiente de Desenvolvimento

- **Node.js:** Versão v18.x ou superior (com `npm`).
- **Java Development Kit (JDK):** JDK 17 (Requerido pelo Gradle do Android).
- **Android SDK:** Build-Tools 34.0.0+, Platform-Tools e Android SDK Command-line Tools.
- **PowerShell:** Versão 5.1 ou superior (para execução de scripts de build automatizados).

6.2 Comandos de Compilação e Automação (npm scripts)

COMANDO NPM	AÇÃO EXECUTADA	RESULTADO GERADO
<code>npm start</code>	Executa o servidor PowerShell local em <code>server.ps1</code> na porta 8080.	Aplicação acessível localmente via HTTP.
<code>npm run build:cap</code>	Copia arquivos (<code>index.html</code> , <code>js/</code> , <code>css/</code>) para <code>www/</code> e executa <code>npx cap sync android</code> .	Sincronização completa de assets com o projeto Android nativo.
<code>npm run build:apk</code>	Sincroniza assets web e executa o <code>gradlew assembleDebug</code> com as variáveis de ambiente JDK/SDK corretas.	Gera o arquivo LINSORA-Financas.apk na raiz do projeto.
<code>npm test</code>	Executa a suíte completa de testes automatizados E2E via Playwright.	Relatório de validação de testes E2E.

6.3 Variáveis de Ambiente e Chaves de Integração

⚠ **Credenciais e Chaves de API Configuradas:**

- **Google OAuth Client ID:** `465194772971-m698u510n7u3n311cbn8oha0rfn6staq.apps.googleusercontent.com` (Configurado em `capacitor.config.json` e `google-auth.js`).
- **Supabase Project URL:** `https://npsynxedtzjixnlyzbsu.supabase.co` (Configurado em `js/supabase-client.js`).
- **Supabase Anon Key:** Configurada no módulo `supabase-client.js` para acesso seguro via RLS.